

Quality Metrics

Campus Equipment Booking & Return System

All figures below come directly from actual pytest/coverage runs against this repository - none are estimated.

1. Test execution summary

Metric	Value
Total automated test cases	127 (88 unit + 27 integration + 12 E2E)
Executed	115 (all unit + integration)
Blocked	12 (all E2E - no installable browser in the development sandbox; see Test Summary Report)
Passed	115
Failed (at time of writing)	0
Pass rate (of executed)	100% (115/115)
Automated test percentage	100%

2. Coverage

Module	Branch coverage
app/services/booking_service.py	99%
app/auth.py	100%
app/models.py	93%
app/__init__.py	92%
app/routes.py	78%
Overall (app/)	87%

Interpretation: the brief's 80% branch-coverage target is met and exceeded on the core business logic (booking_service.py, auth.py), which is where the real risk lives (booking rules, conflict detection, permissions). routes.py's lower figure is expected: it's mostly thin glue (parse form, call service, redirect/flash), and the remaining uncovered paths are largely error-flash branches exercised more naturally by the Selenium E2E suite (once run in CI) than by route-level integration tests aimed at business outcomes.

3. Defect metrics

Metric	Value	Calculation
Defects discovered	4 (DEF-001 to DEF-004)	From docs/defect_log.md
Defects by severity	High: 1, Medium: 1, Low: 2	From defect log
Defects by status	Closed: 4, Open: 0	From defect log
Defect density	4 / 1.607 KLOC $\approx$ 2.5 defects/KLOC	1607 lines in app/ (measured via wc -l)
Defect Removal Efficiency (DRE)	100% (4 found / (4 found + 0 escaped))	No defects have surfaced post-deployment because the app has not been deployed; this figure will only be meaningful after release
Defects found vs. escaped	4 found during testing, 0 escaped	Same caveat
Regression caught during development	1/1 (DEF-002, found by test_it305 on its very first run)	This is the clearest evidence the test-writing process itself found value, not just confirmed existing behaviour

Interpretation: unlike a from-scratch build's deliberately-introduced demonstration regressions, DEF-002 here was a *pre-existing, real* bug in the delivered codebase, caught the first time an integration test actually exercised the full approve -> activate -> return lifecycle end to end. That it survived undetected until a genuine end-to-end integration test ran is itself informative: unit tests alone (which mocked or

isolated individual functions) would not necessarily have caught a self-counting bug that only manifests when a booking's own row is present in the database at the time its own availability is re-checked - this is exactly the kind of defect integration-level testing exists to catch, versus unit-level testing.

4. What the metrics imply about product quality

Before this testing pass, the application had a defect that would have made a common real-world scenario (a single unit of equipment being approved for the only student who requested it) silently fail in production, plus a UI bug that hid admin navigation from admins. Neither would necessarily have been caught by casual manual testing, since a developer testing the happy path with equipment that has multiple units available would not encounter the `approve_booking` bug at all. Both are now fixed and covered by regression tests. The remaining open question is whether the same is true of the browser-driven layer (E2E) - integration tests prove the *routes* work, but a Selenium run against a real rendered page is the only way to confirm the *actual user experience* matches, which has not yet been executed (see Test Summary Report recommendation).